

I/O Multiplexing and Control Module (IOMM)

This chapter describes the I/O Multiplexing and Control Module (IOMM).

Topic	Page
4.1 Overview	240
4.2 Main Features of I/O Multiplexing Module (IOMM)	240
4.3 Control of Multiplexed Outputs	240
4.4 Control of Multiplexed Inputs	241
4.5 Control of Special Multiplexed Options	242
4.6 Safety Features	251
4.7 IOMM Registers	252
4.8 Output Multiplexing and Control.....	265

4.1 Overview

This chapter describes the overall features of the module that controls the I/O multiplexing on the device. The mapping of control registers to multiplexing options is specified in [Section 4.8](#).

4.2 Main Features of I/O Multiplexing Module (IOMM)

The IOMM contains memory-mapped registers (MMR) that control device-specific multiplexed functions. The safety and diagnostic features of the IOMM are:

- Kicker mechanism to protect the MMRs from accidental writes
- Error indication for access violations

4.3 Control of Multiplexed Outputs

The signal multiplexing controlled by each memory-mapped control register (PINMMRn) is described in [Table 4-21](#). Each byte in the PINMMRs control the functionality output on a single terminal. Consider the following example for the PINMMR10 control register.

Figure 4-1. PINMMR10 Control Register, Address = FFFF EB38h

31	26	25	24
Reserved		EMIF DATA[2]	EMIF DATA[2]
RWP-0		R/WP-0	R/WP-1
23	18	17	16
Reserved		N2HET2[7]	Reserved
RWP-0		R/WP-0	R/WP-1
15	10	9	8
Reserved		EMIF DATA[3]	EMIF DATA[3]
RWP-0		R/WP-0	R/WP-1
7	3	2	1
Reserved		RMII_RX_ER	MII_RX_ER
RWP-0		R/WP-0	R/WP-1
			0
			AD1 EVT
			R/WP-1

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privileged mode only; -n = value after reset

- Consider the multiplexing controlled by PINMMR10[23–16]. These bits control the multiplexing between the EMIF_nCS[0] and N2HET2[7] on the ball N17 of the 337BGA package for this device. The default function on the N17 ball is EMIF_nCS[0]. This is dictated by bit 16 of the PINMMR10 register being set.
- If the application wants to use N17 as an N2HET2[7] signal, then bit 16 of PINMMR10 must be cleared and bit 18 must be set.
- Each feature of the output function is determined by the function selected to be output on a terminal.

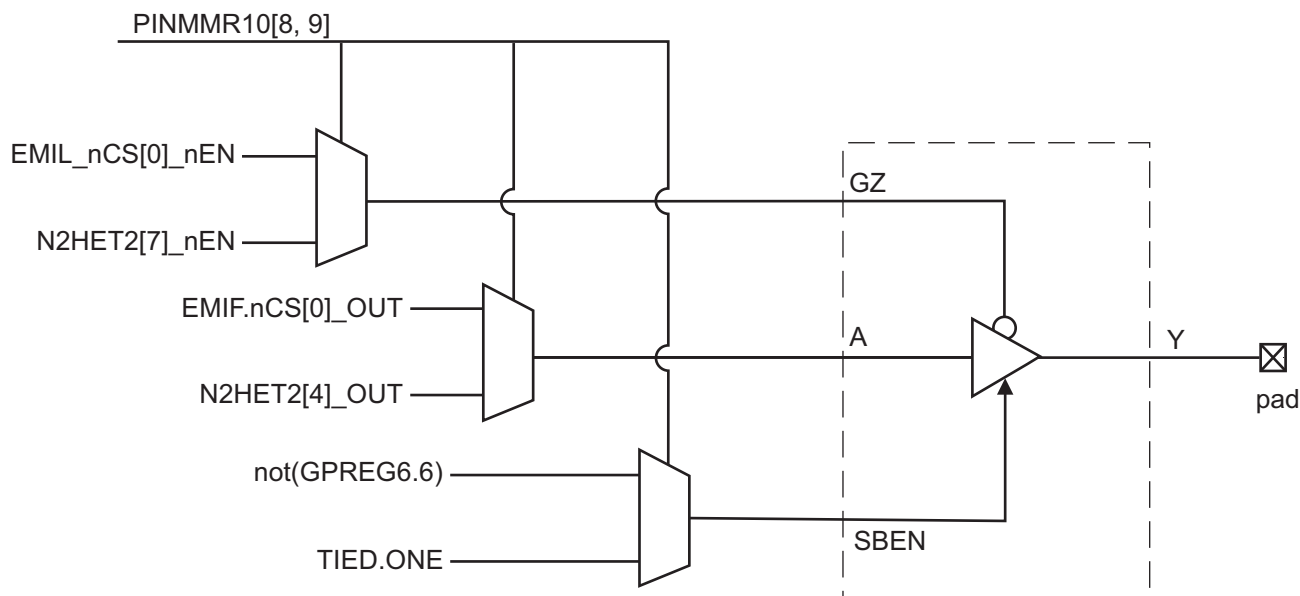
For example, the ball N17 on the 337BGA package is driven by an output buffer with an 8mA drive strength and which supports the adaptive impedance control mode for reduced electromagnetic emissions. This output buffer has the following signals: A (signal to be output), GZ (output enable), SBEN (Standard Buffer Enable) and LPM (Low Power Mode). Each of these signals is an output of a multiplexor which allows the selected function to control all available features of the output buffer.

- The PINMMR control registers are used to implement a one-hot encoding scheme for selecting the multiplexed function.
 - For example, for the N17 ball on the 337BGA package for this device at least one out of bit 16 or bit 18 must be set.
 - If the application clears both bits 16 and 18, then the default function, EMIF_nCS[0], will be selected for output on N17.
 - If the application sets both bits 16 and 18, then the default function will be selected for output on N17.
 - If the application sets one or more reserved bit(s) within the byte 23–16, then the default function

will be selected for output on N17.

Figure 4-2 shows the multiplexing between the output functions for the N17 ball. This terminal uses an 8mA low-EMI output buffer.

Figure 4-2. Output Multiplexing Example



4.4 Control of Multiplexed Inputs

The microcontrollers are available in two package options – 337-ball grid array (BGA) and 144-pin Quad Flat Pack (QFP). Input multiplexing is not required for the 337BGA package. For the 144QFP package, some signals have a multiplexor implemented on their input side. The selection between the two input paths is done by a combination of two control registers. These registers are described in Table 4-1.

Table 4-1. Input Multiplexing on 144QFP Parts

Signal	144QFP Dedicated Input Pin #	144QFP Multiplexed Input Pin #	A, Output Control for Muxed Pin	B, Input Control for Muxed Pin
SPI4SIMO	–	30	PINMMR5[9]	PINMMR23[16]
SPI4SOMI	–	31	PINMMR5[17]	PINMMR23[24]
SPI4CLK	–	25	PINMMR5[1]	PINMMR23[8]
SPI4NENA	–	23	PINMMR4[17]	PINMMR24[0]
SPI4NCS[0]	–	24	PINMMR4[25]	PINMMR24[8]
N2HET1[17]	–	130	PINMMR20[17]	PINMMR24[16]
N2HET1[19]	–	40	PINMMR8[9]	PINMMR24[24]
N2HET1[23]	–	96	PINMMR12[17]	PINMMR25[8]
N2HET1[25]	–	37	PINMMR7[9]	PINMMR25[16]
N2HET1[27]	–	4	PINMMR0[26]	PINMMR25[24]
N2HET1[29]	–	3	PINMMR0[18]	PINMMR26[0]
N2HET1[31]	–	54	PINMMR9[10]	PINMMR26[8]
GIOB[2]	142	55	PINMMR29[16]	PINMMR29[16]

For signals with a “–” in the column for dedicated input pin #, these signals do have a dedicated pad on the die. The default input for these signals comes from the dedicated pad. To be able to drive in the input from the multiplexed pins on the 144QFP package, the IOMM registers need to be configured.

Table 4-1 shows two controls for each of these signals: A and B.

A indicates the control register bit to be set to enable the corresponding signal to be output on the multiplexed pin.

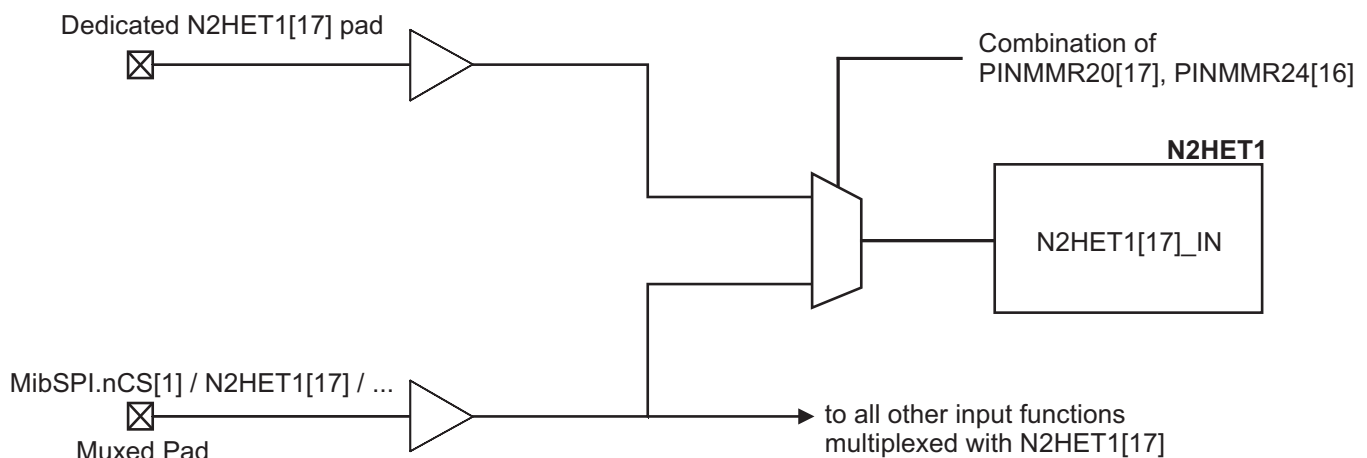
B is another control register bit that is used to select between the dedicated input terminal and the multiplexed pin.

- The input to a module comes from the **dedicated** pad when the condition **[not(A) or (A AND B)] = TRUE**.
- The input to a module comes from the **multiplexed** pin when the condition **[A and not(B)] = TRUE**.

NOTE: Inputs are broadcast to all multiplexed functions

The input signals are broadcast to all modules hooked up to a terminal. The application must ensure that modules that are not being used in the application do not react to a change on their input functions. For example, a GIO signal toggle can trigger an interrupt request, when the application actually is using the function multiplexed with this GIO signal.

Figure 4-3. Input Multiplexing Example



4.5 Control of Special Multiplexed Options

Several of the PINMMR registers are used to control specific functions on this microcontroller.

4.5.1 Control of SDRAM clock (EMIF_CLK)

PINMMR29[8] is set by default. This blocks the EMIF SDRAM clock signal (EMIF_CLK) from being output from the microcontroller. If the EMIF is used to connect to an external SDRAM module, then the application must enable the SDRAM clock output by clearing the PINMMR29[8] bit.

4.5.2 Control for other EMIF Outputs

There are some EMIF address and control signals that are multiplexed with N2HET2 signals. For applications that require the use of these N2HET2 signals, it is inconvenient if the EMIF starts driving these address and control signals as output after reset is released and before the application can configure the I/O Multiplexing Module registers. Therefore, these EMIF signals are blocked from being output by default. This is done by using the bit 31 of the system module General Purpose Control Register (GPREG1). This bit is cleared (0) by default. In this condition, these EMIF/N2HET2 terminals are configured as inputs and pulled down. An application that requires the EMIF functionality must set the GPREG1[31] bit. This causes the EMIF address and control signals to then be output on the EMIF/N2HET2 terminals when the EMIF functionality is selected via the IOMM output multiplexing control registers.

4.5.3 Control for Fast Mode for Synchronous and Asynchronous EMIF

The EMIF on this microcontroller implements a "fast mode" for both the synchronous and asynchronous interfaces. There are some critical timing paths on read accesses to external asynchronous as well as synchronous memories. The fast mode allows the clock to the capture flops inside the EMIF module to be pushed in order to capture the incoming data correctly. The fast mode for asynchronous interface is enabled by setting PINMMR29[18] and the fast mode for the synchronous interface is enabled by setting PINMMR29[17].

4.5.4 Control of Ethernet Controller Mode

PINMMR29[24] is set by default. This bit is used to enable the Reduced Media Independent Interface of the Ethernet controller. If the application desires to use the Media Independent Interface of the Ethernet controller, then the PINMMR29[24] must be cleared.

4.5.5 Control of ADC Trigger Events

The microcontrollers contain two Analog-to-Digital Converter modules: ADC1 and ADC2. The ADC conversions can be started using a rising or falling or both edges as the trigger event. Both the ADC modules support up to eight event trigger inputs. There are two sets of these 8 inputs for each ADC. The option for each of these 8 inputs are controlled by registers in the I/O multiplexing module as shown in [Table 4-2](#) and [Table 4-3](#).

Table 4-2. ADC1 Trigger Event Selection

Group Source Select, G1SRC, G2SRC or EVSRC	Event #	Trigger Event Signal				
		PINMMR30[0] = 1 (default)	PINMMR30[0] = 0 and PINMMR30[1] = 1			
			Option A	Control for Option A	Option B	Control for Option B
000	1	AD1EVT	AD1EVT	—	AD1EVT	—
001	2	N2HET1[8]	N2HET2[5]	PINMMR30[8] = 1	ePWM_B	PINMMR30[8] = 0 and PINMMR30[9] = 1
010	3	N2HET1[10]	N2HET1[27]	—	N2HET1[27]	—
011	4	RTI Compare 0 Interrupt	RTI Compare 0 Interrupt	PINMMR30[16] = 1	ePWM_A1	PINMMR30[16] = 0 and PINMMR30[17] = 1
100	5	N2HET1[12]	N2HET1[17]	—	N2HET1[17]	—
101	6	N2HET1[14]	N2HET1[19]	PINMMR30[24] = 1	N2HET2[1]	PINMMR30[24] = 0 and PINMMR30[25] = 1
110	7	GIOB[0]	N2HET1[11]	PINMMR31[0] = 1	ePWM_A2	PINMMR31[0] = 0 and PINMMR31[1] = 1
111	8	GIOB[1]	N2HET2[13]	PINMMR31[8] = 1	ePWM_AB	PINMMR31[8] = 0 and PINMMR31[9] = 1

Table 4-3. ADC2 Trigger Event Selection

Group Source Select, G1SRC, G2SRC or EVSRC	Event #	Trigger Event Signal				
		PINMMR30[0] = 1 (default)	PINMMR30[0] = 0 and PINMMR30[1] = 1			
			Option A	Control for Option A	Option B	Control for Option B
000	1	AD2EVT	AD1EVT	—	AD1EVT	—

Table 4-3. ADC2 Trigger Event Selection (continued)

001	2	N2HET1[8]	N2HET2[5]	PINMMR31[16] = 1	ePWM_B	PINMMR31[16] = 0 and PINMMR31[17] = 1
010	3	N2HET1[10]	N2HET1[27]	—	N2HET1[27]	—
011	4	RTI Compare 0 Interrupt	RTI Compare 0 Interrupt	PINMMR31[24] = 1	ePWM_A1	PINMMR31[24] = 0 and PINMMR31[25] = 1
100	5	N2HET1[12]	N2HET1[17]	—	N2HET1[17]	—
101	6	N2HET1[14]	N2HET1[19]	PINMMR32[0] = 1	N2HET2[1]	PINMMR32[0] = 0 and PINMMR32[1] = 1
110	7	GIOB[0]	N2HET1[11]	PINMMR32[8] = 1	ePWM_A2	PINMMR32[8] = 0 and PINMMR32[9] = 1
111	8	GIOB[1]	N2HET2[13]	PINMMR32[16] = 1	ePWM_AB	PINMMR32[16] = 0 and PINMMR32[17] = 1

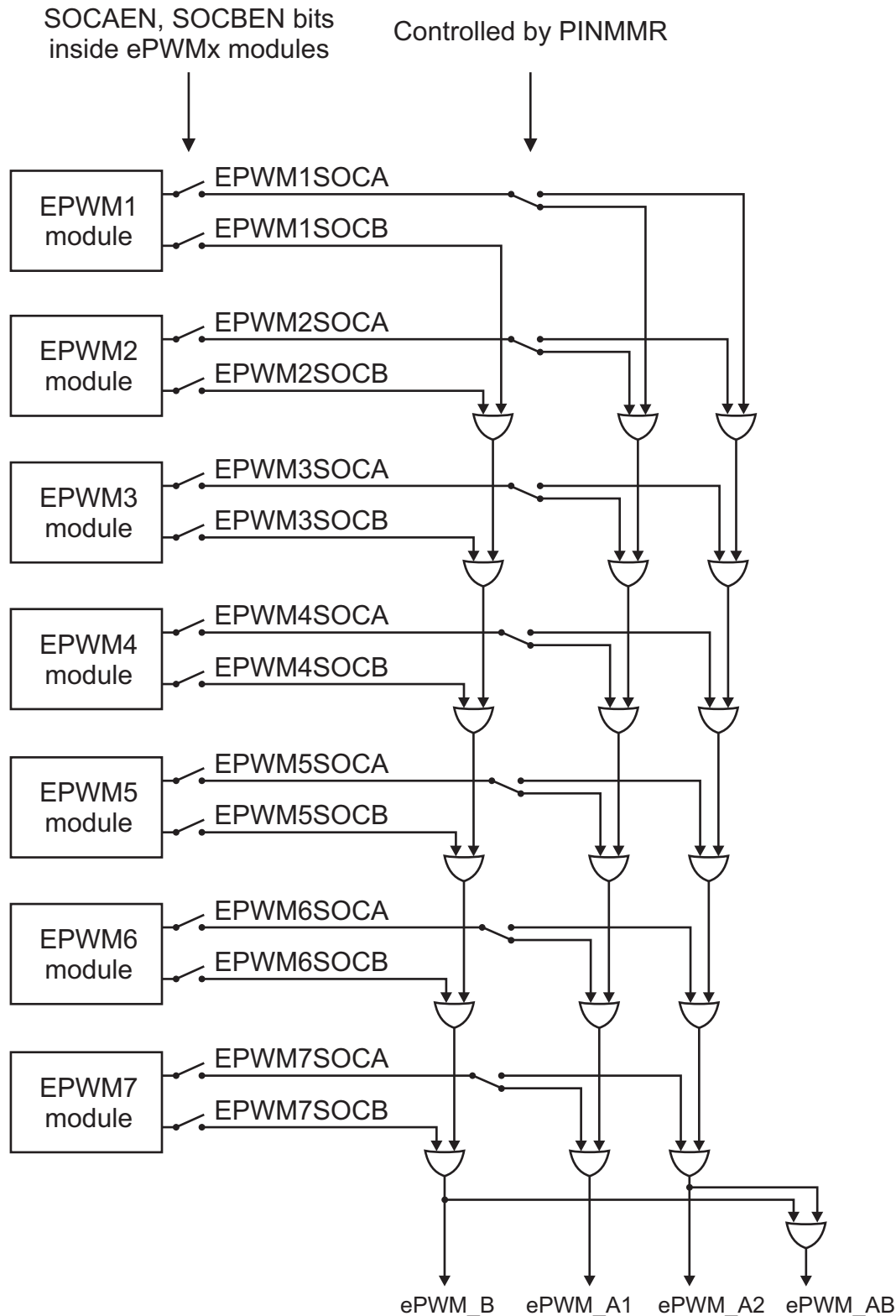
4.5.6 Control for ADC Event Trigger Signal Generation from ePWMx Modules

This microcontroller implements 7 ePWM modules. Each of these modules generate two outputs, SOCA (Start Of Conversion) and SOCB, for use in triggering the on-chip ADC modules. Registers from the I/O multiplexing module are used to control the logic for generation of the ePWM_A1, ePWM_A2, ePWM_AB, and ePWM_B signals from these ePWMx_SOCA and ePWMx_SOCB signals.

The logic equations for the 4 outputs from the combinational logic shown in [Figure 4-4](#) are:

- B = SOC1B or SOC2B or SOC3B or SOC4B or SOC5B or SOC6B or SOC7B
- A1 = [SOC1A and not(SOC1A_SEL)]
[SOC2A and not(SOC2A_SEL)]
[SOC3A and not(SOC3A_SEL)]
[SOC4A and not(SOC4A_SEL)]
[SOC5A and not(SOC5A_SEL)]
[SOC6A and not(SOC6A_SEL)]
[SOC7A and not(SOC7A_SEL)]
- A2 = [SOC1A and SOC1A_SEL]
[SOC2A and SOC2A_SEL]
[SOC3A and SOC3A_SEL]
[SOC4A and SOC4A_SEL]
[SOC5A and SOC5A_SEL]
[SOC6A and SOC6A_SEL]
[SOC7A and SOC7A_SEL]
- AB = B or A2

Figure 4-4. ADC Trigger Event Signal Generation from ePWMx



The SOCxA_SEL signals used in the above logic equations are generated using registers in the I/O multiplexing module.

- PINMMR35[0] defines the value of SOC1A_SEL. This bit is set by default and can be cleared by the application.
- PINMMR35[8] defines the value of SOC2A_SEL. This bit is set by default and can be cleared by the application.
- PINMMR35[16] defines the value of SOC3A_SEL. This bit is set by default and can be cleared by the application.
- PINMMR35[24] defines the value of SOC4A_SEL. This bit is set by default and can be cleared by the application.
- PINMMR36[0] defines the value of SOC5A_SEL. This bit is set by default and can be cleared by the application.
- PINMMR36[8] defines the value of SOC6A_SEL. This bit is set by default and can be cleared by the application.
- PINMMR36[16] defines the value of SOC7A_SEL. This bit is set by default and can be cleared by the application.

4.5.7 Control for Generating Interrupt Upon External Fault Indication to N2HETx

The N2HET module on this microcontroller allows the application to selectively disable any PWM output from the N2HET module whenever a fault condition is indicated to the N2HET. This fault condition is input to the N2HET module via the N2HETx_PIN_nDIS input signal. It is important for the CPU to be notified with an interrupt whenever this fault condition is indicated to the N2HET module.

The PIN_nDIS signal for the N2HET1 module comes from the GIOA[5] terminal, and the application can enable the interrupt generation for whenever the GIOA[5] terminal is driven low.

The PIN_nDIS signal for the N2HET2 module comes from the MibSPI3_nCS[0] / AD2EVT terminal. These signals do not offer the capability of generating an interrupt when they are driven low. Therefore, the input from this terminal can optionally be connected to the GIOB[2] input. This connection is enabled by a register in the I/O multiplexing module. This is the PINMMR29[16].

4.5.8 Control for Enabling Clocks to ePWMx Modules

The VCLK4 domain is used as the clock for the ePWMx modules. This clock can be individually gated off when one or more of the ePWMx modules are not being used.

- PINMMR37[8] is set by default and enables the clock to the ePWM1. The application can clear this bit to disable the clock to the ePWM1.
- PINMMR37[16] is set by default and enables the clock to the ePWM2. The application can clear this bit to disable the clock to the ePWM2.
- PINMMR37[24] is set by default and enables the clock to the ePWM3. The application can clear this bit to disable the clock to the ePWM3.
- PINMMR38[0] is set by default and enables the clock to the ePWM4. The application can clear this bit to disable the clock to the ePWM4.
- PINMMR38[8] is set by default and enables the clock to the ePWM5. The application can clear this bit to disable the clock to the ePWM5.
- PINMMR38[16] is set by default and enables the clock to the ePWM6. The application can clear this bit to disable the clock to the ePWM6.
- PINMMR38[24] is set by default and enables the clock to the ePWM7. The application can clear this bit to disable the clock to the ePWM7.

4.5.9 Control for Enabling Clocks to eCAPx Modules

The VCLK4 domain is used as the clock for the eCAPx modules. This clock can be individually gated off when one or more of the eCAPx modules are not being used.

- PINMMR39[0] is set by default and enables the clock to the eCAP1. The application can clear this bit to disable the clock to the eCAP1.
- PINMMR39[8] is set by default and enables the clock to the eCAP2. The application can clear this bit to disable the clock to the eCAP2.
- PINMMR39[16] is set by default and enables the clock to the eCAP3. The application can clear this bit to disable the clock to the eCAP3.
- PINMMR39[24] is set by default and enables the clock to the eCAP4. The application can clear this bit to disable the clock to the eCAP4.
- PINMMR40[0] is set by default and enables the clock to the eCAP5. The application can clear this bit to disable the clock to the eCAP5.
- PINMMR40[8] is set by default and enables the clock to the eCAP6. The application can clear this bit to disable the clock to the eCAP6.

4.5.10 Control for Enabling Clocks to eQEPx Modules

The VCLK4 domain is used as the clock for the eQEPx modules. This clock can be individually gated off when one or more of the eQEPx modules are not being used.

- PINMMR40[16] is set by default and enables the clock to the eQEP1. The application can clear this bit to disable the clock to the eQEP1.
- PINMMR40[24] is set by default and enables the clock to the eQEP2. The application can clear this bit to disable the clock to the eQEP2.

4.5.11 Control for Synchronizing Time Bases for All ePWMx Modules

The ePWMx modules implement a mechanism that allows their time bases to be synchronized. This is done by using a signal called TBCLKSYNC, which is a common input to all the ePWMx modules. This TBCLKSYNC is generated by a register bit in the I/O multiplexing module. PINMMR37[1] is the TBCLKSYNC signal. This bit is clear (0) by default.

When TBCLKSYNC = 0, the time-base clock of all ePWMx modules is stopped. This is the default condition.

When TBCLKSYNC = 1, the time-base clocks of all ePWMx modules are started aligned to the rising edge of the TBCLKSYNC signal.

The correct procedure for enabling and synchronizing the time-base clocks of all the ePWMx modules is:

1. Enable the clocks to the desired individual ePWMx modules if they have been disabled
2. Set TBCLKSYNC = 0. This will stop the time-base clocks of any enabled ePWMx module.
3. Configure the time-base clock prescaler values and desired ePWM modes.
4. Set TBCLKSYNC = 1.

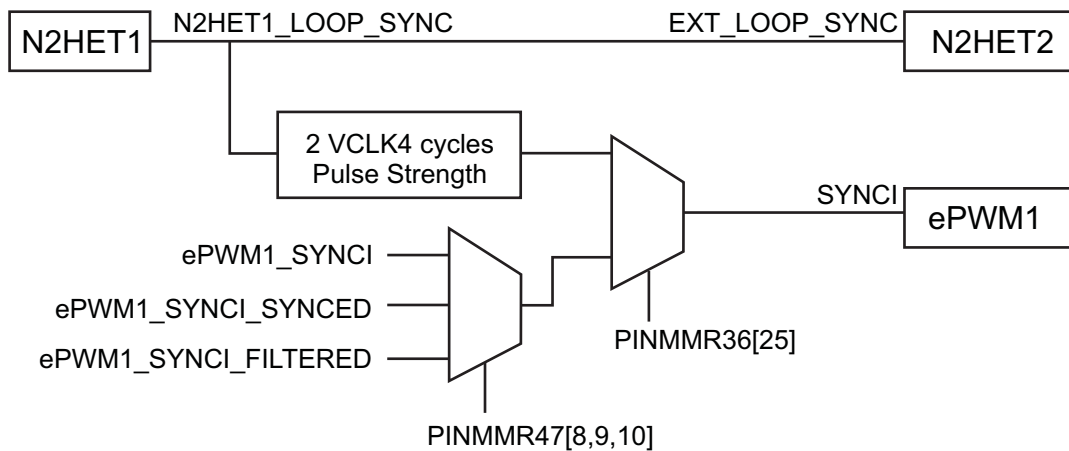
4.5.12 Control for Synchronizing all ePWMx Modules to N2HET1 Module Time-Base

Some applications require a synchronized time base for all PWM signals generated by the microcontroller. The N2HET1 module uses a time base that is created by configuring the high-resolution and loop-resolution prescalers in the N2HET1 module control registers. The N2HET1 module outputs the loop-resolution clock signal (N2HET1_LOOP_SYNC) so that other timer modules on the microcontroller can use it to synchronize their time bases to the N2HET1 loop-resolution clock.

There is a dedicated connection between the N2HET1 and N2HET2 modules, which allows the N2HET2 to use the N2HET1_LOOP_SYNC signal to synchronize its own time base to that of N2HET1.

The seven ePWMx modules can also optionally use the N2HET1_LOOP_SYNC for their time-base synchronization using a specially designed scheme.

Figure 4-5. Synchronizing ePWMx Modules to N2HET1 Time-Base



PINMMR36[25] is used to select between the ePWM1_SYNCI and the stretched N2HET1_LOOP_SYNC signals.

If PINMMR36[25] = 0, the SYNCI input to the ePWM1 comes from the ePWM1_SYNCI device input terminal. This is the default connection.

If PINMMR36[25] = 1, the SYNCI input to the ePWM1 comes from the pulse-stretched N2HET1_LOOP_SYNC signal.

4.5.13 Control for Input Connections to ePWMx Modules

The ePWMx modules take the following signals as input:

- ePWM1_SYNCl: external time-base input to the ePWMx
- nTZ1 through nTZ6: trip-zone inputs to the ePWMx

Of the six trip-zone inputs, three are input from device terminals while the other three are connected to internal error events. Registers from the I/O multiplexing module are used to control various aspects of these input connections to the ePWMx modules.

Table 4-4. Controls for ePWMx Inputs

Input Signal	Control for Asynchronous Input (default connection)	Control for Double-VCLK4-Synchronized Input	Control for Double-VCLK4-Synchronized and 6-VCLK4-Filtered Input
nTZ1	PINMMR46[16] = 1	PINMMR46[16] = 0 and PINMMR46[17] = 1	PINMMR46[17:16] = 00 and PINMMR46[18] = 1
nTZ2	PINMMR46[24] = 1	PINMMR46[24] = 0 and PINMMR46[25] = 1	PINMMR46[25:24] = 00 and PINMMR46[26] = 1
nTZ3	PINMMR47[0] = 1	PINMMR47[0] = 0 and PINMMR47[1] = 1	PINMMR47[1:0] = 00 and PINMMR47[2] = 1
ePWM1_SYNCl	PINMMR47[8] = 1	PINMMR47[8] = 0 and PINMMR47[9] = 1	PINMMR47[9:8] = 00 and PINMMR47[10] = 1

Of the three internal error events for the trip-zone inputs, nTZ4 is connected to the eQEPx error output signal. There are two eQEP modules on this microcontroller, and registers in the I/O multiplexing module are used to allow a flexible scheme for the connection between the eQEPx error signal and the nTZ4 inputs of the ePWMx modules.

Table 4-5. Controls for eQEPx_ERROR Connection to ePWMx nTZ4 Inputs

ePWMx Module	Control for nTZ4 = not(eQEP1ERR or eQEP2ERR) (default connection)	Control for nTZ4 = not(eQEP1ERR)	Control for nTZ4 = not(eQEP2ERR)
ePWM1	PINMMR41[0] = 1	PINMMR41[0] = 0 and PINMMR41[1] = 1	PINMMR41[1:0] = 00 and PINMMR41[2] = 1
ePWM2	PINMMR41[8] = 1	PINMMR41[8] = 0 and PINMMR41[9] = 1	PINMMR41[9:8] = 00 and PINMMR41[10] = 1
ePWM3	PINMMR41[16] = 1	PINMMR41[16] = 0 and PINMMR41[17] = 1	PINMMR41[17:16] = 00 and PINMMR41[18] = 1
ePWM4	PINMMR41[24] = 1	PINMMR41[24] = 0 and PINMMR41[25] = 1	PINMMR41[25:24] = 00 and PINMMR41[26] = 1
ePWM5	PINMMR42[0] = 1	PINMMR42[0] = 0 and PINMMR42[1] = 1	PINMMR42[1:0] = 00 and PINMMR42[2] = 1
ePWM6	PINMMR42[8] = 1	PINMMR42[8] = 0 and PINMMR42[9] = 1	PINMMR42[9:8] = 00 and PINMMR42[10] = 1
ePWM7	PINMMR42[16] = 1	PINMMR42[16] = 0 and PINMMR42[17] = 1	PINMMR42[17:16] = 00 and PINMMR42[18] = 1

4.5.14 Control for Input Connections to eCAPx Modules

Each eCAPx module has a single input from the device terminals. This input can be connected to the eCAPx module in two ways:

1. Double-synchronized using VCLK4, or
2. Double-synchronized using VCLK4 and then filtered through a 6-VCLK4-cycle counter

Registers in the I/O multiplexing module are used to control these input connections for each eCAPx module.

Table 4-6. Controls for eCAPx Inputs

eCAPx Input	Control for Double-VCLK4-Synchronized Input (default connection)	Control for Double-VCLK4-Synchronized and 6-VCLK4-Filtered Input
eCAP1	PINMMR43[0] = 0	PINMMR43[0] = 0 and PINMMR43[1] = 1
eCAP2	PINMMR43[8] = 0	PINMMR43[8] = 0 and PINMMR43[9] = 1
eCAP3	PINMMR43[16] = 0	PINMMR43[16] = 0 and PINMMR43[17] = 1
eCAP4	PINMMR43[24] = 0	PINMMR43[24] = 0 and PINMMR43[25] = 1
eCAP5	PINMMR44[0] = 0	PINMMR44[0] = 0 and PINMMR44[1] = 1
eCAP6	PINMMR44[8] = 0	PINMMR44[8] = 0 and PINMMR44[9] = 1

4.5.15 Control for Input Connections to eQEPx Modules

Each eQEPx module has four inputs from the device terminals. These inputs can be connected to the eQEPx module in two ways:

1. Double-synchronized using VCLK4, or
2. Double-synchronized using VCLK4 and then filtered through a 6-VCLK4-cycle counter

Registers in the I/O multiplexing module are used to control these input connections for each eQEPx module.

Table 4-7. Controls for eQEPx Inputs

eQEPx Input	Control for Double-VCLK4-Synchronized Input (default connection)	Control for Double-VCLK4-Synchronized and 6-VCLK4-Filtered Input
eQEP1A	PINMMR44[16] = 1	PINMMR44[16] = 0 and PINMMR44[17] = 1
eQEP1B	PINMMR44[24] = 1	PINMMR44[24] = 0 and PINMMR44[25] = 1
eQEP1I	PINMMR45[0] = 1	PINMMR45[0] = 0 and PINMMR45[1] = 1
eQEP1S	PINMMR45[8] = 1	PINMMR45[8] = 0 and PINMMR45[9] = 1
eQEP2A	PINMMR45[16] = 1	PINMMR45[16] = 0 and PINMMR45[17] = 1
eQEP2B	PINMMR45[24] = 1	PINMMR45[24] = 0 and PINMMR45[25] = 1
eQEP2I	PINMMR46[0] = 1	PINMMR46[0] = 0 and PINMMR46[1] = 1
eQEP2S	PINMMR46[8] = 1	PINMMR46[8] = 0 and PINMMR46[9] = 1

4.6 Safety Features

The IOMM supports certain safety functions that are designed to prevent unintentional changes to the I/O multiplexing configuration. These are described in the following sections.

4.6.1 Locking Mechanism for Memory-Mapped Registers

The IOMM contains a mechanism to prevent any spurious writes from changing any of the PINMMR values. The PINMMRs are locked by default and after any system reset. None of the PINMMRs can be written under this condition. The application can read any of the IOMM registers regardless of the state of the locking mechanism.

- **Enabling Write Access to the PINMMRs**

To enable write access to the PINMMRs, the CPU must write 0x83e70b13 to the kick0 register followed by a write of 0x95a4f1e0 to the kick1 register.

- **Disabling Write Access to the PINMMRs**

It is recommended to disable write access to the PINMMRs once the I/O multiplexing configuration is completed. This can be done by:

- writing any other data value to either of the kick registers, or
- restarting the unlock sequence by writing 0x83e70b13 to the kick0 register

NOTE: No Error On Write to Locked PINMMRs

There is no error response on any write accesses to the PINMMRs when write access is disabled. None of the PINMMRs change state due to this write.

4.6.2 Error Conditions

The IOMM generates one error signal that is mapped to the Error Signaling Module's Group 1, channel 37. This error signal is generated under either of the following two conditions:

- Address Error – occurs when there is a read or a write access to an un-implemented memory location within the IOMM register frame.
- Protection Error – occurs when the CPU writes to an IOMM register while not in a privileged mode of operation.

4.7 IOMM Registers

[Table 4-8](#) lists the memory-mapped registers for the IOMM. All register offset addresses not listed in [Table 4-8](#) should be considered as reserved locations and the register contents should not be modified. The address offset is specified from the base address of FFFF EA00h.

Table 4-8. IOMM Registers

Offset	Acronym	Register Name	Section
0h	REVISION_REG	Revision Register	Section 4.7.1
20h	ENDIAN_REG	Device Endianness Register	Section 4.7.2
38h	KICK_REG0	Kicker Register 0	Section 4.7.3
3Ch	KICK_REG1	Kicker Register 1	Section 4.7.4
E0h	ERR_RAW_STATUS_REG	Error Raw Status / Set Register	Section 4.7.5
E4h	ERR_ENABLED_STATUS_REG	Error Enabled Status / Clear Register	Section 4.7.6
E8h	ERR_ENABLE_REG	Error Signaling Enable Register	Section 4.7.7
ECh	ERR_ENABLE_CLR_REG	Error Signaling Enable Clear Register	Section 4.7.8
F4h	FAULT_ADDRESS_REG	Fault Address Register	Section 4.7.9
F8h	FAULT_STATUS_REG	Fault Status Register	Section 4.7.10
FCh	FAULT_CLEAR_REG	Fault Clear Register	Section 4.7.11
B10h to BCCh	PINMMR_0 to PINMMR_47	Pin Multiplexing Control Registers	Section 4.7.12

4.7.1 REVISION_REG Register (Offset = 0h) [reset = 4E840102h]

REVISION_REG is shown in [Figure 4-6](#) and described in [Table 4-9](#).

This is a read-only register that provides the revision information about the I/O Multiplexing Module (IOMM).

Figure 4-6. REVISION_REG Register

31	30	29	28	27	26	25	24
REV_SCHEME		RESERVED		REV_MODULE			
R-1h		R-0h		R-E84h			
23	22	21	20	19	18	17	16
REV_MODULE							
R-E84h							
15	14	13	12	11	10	9	8
REV_RTL					REV_MAJOR		
R-0h					R-1h		
7	6	5	4	3	2	1	0
REV_CUSTOM		REV_MINOR					
R-0h		R-2h					

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-9. REVISION_REG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-30	REV_SCHEME	R	1h	Revision Scheme, value = 01.
29-28	RESERVED	R	0h	
27-16	REV_MODULE	R	E84h	Module Id, value = E84h.
15-11	REV_RTL	R	0h	RTL Revision, value = 0.
10-8	REV_MAJOR	R	1h	Major Revision, value = 001.
7-6	REV_CUSTOM	R	0h	Custom Revision, value = 0.
5-0	REV_MINOR	R	2h	Minor Revision, value = 2h.

4.7.2 ENDIAN_REG Register (Offset = 20h) [reset = 0h]

ENDIAN_REG is shown in [Figure 4-7](#) and described in [Table 4-10](#).

This is a read-only register that reflects the state of the device endianness.

Figure 4-7. ENDIAN_REG Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							ENDIAN
R-0h							R-0h

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-10. ENDIAN_REG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	ENDIAN	R	0h	Device endianness 0h = Device is configured in little-endian mode. 1h = Device is configured in big-endian mode.

4.7.3 KICK_REG0 Register (Offset = 38h) [reset = 0h]

KICK_REG0 is shown in [Figure 4-8](#) and described in [Table 4-11](#).

This register forms the first part of the unlock sequence for being able to update the I/O multiplexing control registers (PINMMRn).

Figure 4-8. KICK_REG0 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KICK0																R/W-0h															

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-11. KICK_REG0 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-0	KICK0	R/W	0h	Kicker 0 Register. The value 83E7 0B13h must be written to KICK0 as part of the process to unlock the CPU write access to the PINMMRn registers.

4.7.4 KICK_REG1 Register (Offset = 3Ch) [reset = 0h]

KICK_REG1 is shown in [Figure 4-9](#) and described in [Table 4-12](#).

This register forms the second part of the unlock sequence for being able to update the I/O multiplexing control registers (PINMMRn).

Figure 4-9. KICK_REG1 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KICK1																															
R/W-0h																															

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-12. KICK_REG1 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-0	KICK1	R/W	0h	Kicker 1 Register. The value 95A4 F1E0h must be written to the KICK1 as part of the process to unlock the CPU write access to the PINMMRn registers.

4.7.5 ERR_RAW_STATUS_REG Register (Offset = E0h) [reset = 0h]

ERR_RAW_STATUS_REG is shown in [Figure 4-10](#) and described in [Table 4-13](#).

This register shows the status of the error conditions (before enabling) and allows setting the status. The IOMM module error signal is connected to the device's Error Signaling Module (ESM) group1 channel 37. The application can choose to generate an interrupt whenever this ESM channel flag gets set. This interrupt service routine can then read this Error Raw Status Register to determine the actual cause of the error condition. The Error Raw Status register is also writable by the application in order to test the ESM signaling and interrupt generation mechanism.

Figure 4-10. ERR_RAW_STATUS_REG Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						ADDR_ERR	PROT_ERR
R-0h						R/W-0h	R/W-0h

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-13. ERR_RAW_STATUS_REG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	ADDR_ERR	R/W	0h	Addressing Error Status. An Addressing Error occurs when an unimplemented location inside the IOMM register frame is accessed. 0h (W) = Writing 0 has no effect. 0h (R) = Addressing Error has not occurred. 1h (W) = Addressing Error status is set. 1h (R) = Addressing Error has been detected.
0	PROT_ERR	R/W	0h	Protection Error Status. A Protection Error occurs when any control register inside the IOMM is written in the CPU's user mode of operation. 0h (W) = Writing 0 has no effect. 0h (R) = Protection Error has not occurred. 1h (W) = Protection Error status is set. 1h (R) = Protection Error has been detected.

4.7.6 ERR_ENABLED_STATUS_REG Register (Offset = E4h) [reset = 0h]

ERR_ENABLED_STATUS_REG is shown in [Figure 4-11](#) and described in [Table 4-14](#).

This register shows the status of the error conditions and allows clearing of the error status.

Figure 4-11. ERR_ENABLED_STATUS_REG Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						ENABLED_ADDR_ERR	ENABLED_PROT_ERR
R-0h						R/W-0h	R/W-0h

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-14. ERR_ENABLED_STATUS_REG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	ENABLED_ADDR_ERR	R/W	0h	Addressing Error Signaling Enable and Status Clear 0h (W) = Writing 0 has no effect. 0h (R) = Addressing Error Signaling is disabled. 1h (W) = Addressing Error status is cleared. 1h (R) = Addressing Error Signaling is enabled.
0	ENABLED_PROT_ERR	R/W	0h	Protection Error Signaling Enable and Status Clear 0h (W) = Writing 0 has no effect. 0h (R) = Protection Error Signaling is disabled. 1h (W) = Protection Error status is cleared. 1h (R) = Protection Error Signaling is enabled.

4.7.7 ERR_ENABLE_REG Register (Offset = E8h) [reset = 0h]

ERR_ENABLE_REG is shown in [Figure 4-12](#) and described in [Table 4-15](#).

This register shows the interrupt enable status and allows enabling of the interrupts.

Figure 4-12. ERR_ENABLE_REG Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						ADDR_ERR_EN	PROT_ERR_EN
R-0h						R/W-0h	R/W-0h

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-15. ERR_ENABLE_REG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	ADDR_ERR_EN	R/W	0h	Addressing Error Signaling Enable 0h (W) = Writing 0 has no effect. 0h (R) = Addressing Error signaling is disabled. 1h (W) = Addressing Error signaling is enabled. 1h (R) = Addressing Error signaling is enabled.
0	PROT_ERR_EN	R/W	0h	Protection Error Signaling Enable 0h (W) = Writing 0 has no effect. 0h (R) = Protection Error signaling is disabled. 1h (W) = Protection Error signaling is enabled. 1h (R) = Protection Error signaling is enabled.

4.7.8 ERR_ENABLE_CLR_REG Register (Offset = ECh) [reset = 0h]

ERR_ENABLE_CLR_REG is shown in [Figure 4-13](#) and described in [Table 4-16](#).

This register shows the error signaling enable status and allows disabling of the error signaling.

Figure 4-13. ERR_ENABLE_CLR_REG Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						ADDR_ERR_EN_CLR	PROT_ERR_EN_CLR
R-0h						R/W-0h	R/W-0h

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-16. ERR_ENABLE_CLR_REG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	ADDR_ERR_EN_CLR	R/W	0h	Addressing Error Signaling Enable Clear 0h (W) = Writing 0 has no effect. 0h (R) = Addressing Error signaling is disabled. 1h (W) = Addressing Error signaling is disabled. 1h (R) = Addressing Error signaling is enabled.
0	PROT_ERR_EN_CLR	R/W	0h	Protection Error Signaling Enable Clear 0h (W) = Writing 0 has no effect. 0h (R) = Protection Error signaling is disabled. 1h (W) = Protection Error signaling is disabled. 1h (R) = Protection Error signaling is enabled.

4.7.9 FAULT_ADDRESS_REG Register (Offset = F4h) [reset = 0h]

FAULT_ADDRESS_REG is shown in [Figure 4-14](#) and described in [Table 4-17](#).

This register holds the address of the first fault transfer.

Figure 4-14. FAULT_ADDRESS_REG Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED								FAULT_ADDR							
R-0h								R/W-0h							

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-17. FAULT_ADDRESS_REG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R	0h	
8-0	FAULT_ADDR	R/W	0h	Fault Address. The fault address offset in case of an address error or a protection error condition.

4.7.10 FAULT_STATUS_REG Register (Offset = F8h) [reset = 0h]

FAULT_STATUS_REG is shown in [Figure 4-15](#) and described in [Table 4-18](#).

This register holds the status and attributes of the first fault transfer.

Figure 4-15. FAULT_STATUS_REG Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED				FAULT_ID				FAULT_MSTID							
R-0h				R-0h				R-0h							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED				FAULT_PRIVID				RESERVED				FAULT_TYPE			
R-0h				R-0h				R-0h				R-0h			

LEGEND: RW = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-18. FAULT_STATUS_REG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	R	0h	
27-24	FAULT_ID	R	0h	Faulting Transaction ID
23-16	FAULT_MSTID	R	0h	ID of Master that initiated the faulting transaction
15-13	RESERVED	R	0h	
12-9	FAULT_PRIVID	R	0h	Faulting Privilege ID
8-6	RESERVED	R	0h	
5-0	FAULT_TYPE	R	0h	Type of fault detected. 0h = No fault 1h = User execute fault 2h = User write fault 4h = User read fault 8h = Supervisor execute fault 10h = Supervisor write fault 20h = Supervisor read fault

4.7.11 FAULT_CLEAR_REG Register (Offset = FCh) [reset = 0h]

FAULT_CLEAR_REG is shown in [Figure 4-16](#) and described in [Table 4-19](#).

This register allows the application to clear the current fault so that another can be captured when 1 is written to this register.

Figure 4-16. FAULT_CLEAR_REG Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							FAULT_CLEAR
R-0h							R/W-0h

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-19. FAULT_CLEAR_REG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	FAULT_CLEAR	R/W	0h	Fault Clear 0h (W) = Writing 0 has no effect. 0h (R) = Current value of the FAULT_CLEAR bit is 0. 1h (W) = Writing a 1 clears the current fault. 1h (R) = Current value of the FAULT_CLEAR bit is 1.

4.7.12 PINMMR_0 to PINMMR_47 Register (Offset = B10h to BCCh) [reset = 1010101h]

PINMMR_0 to PINMMR_47 is shown in [Figure 4-17](#) and described in [Table 4-20](#).

These registers control the multiplexing of the functionality available on each pad as well as some special multiplexing schemes for specific functionalities on the microcontroller. There are 48 such registers PINMMR0 through PINMMR47. Each 8-bit field of a PINMMR register controls the functionality of a single ball/pin. The mapping between the PINMMRx control registers and the functionality selected on a given terminal is defined in .

Figure 4-17. PINMMR_0 to PINMMR_47 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PINMMRx_31_24								PINMMRx_23_16							
R/W-1h								R/W-1h							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINMMRx_15_8								PINMMRx_7_0							
R/W-1h								R/W-1h							

LEGEND: R/W = Read/Write; R = Read only; W1toCl = Write 1 to clear bit; -n = value after reset

Table 4-20. PINMMR_0 to PINMMR_47 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	PINMMRx_31_24	R/W	1h	Each of these byte-fields control the functionality on a given ball/pin. Please refer to Output Multiplexing and Control table for a list of multiplexed signals. This list is sorted by the control register used.
23-16	PINMMRx_23_16	R/W	1h	Each of these byte-fields control the functionality on a given ball/pin. Please refer to Output Multiplexing and Control table for a list of multiplexed signals. This list is sorted by the control register used.
15-8	PINMMRx_15_8	R/W	1h	Each of these byte-fields control the functionality on a given ball/pin. Please refer to Output Multiplexing and Control table for a list of multiplexed signals. This list is sorted by the control register used.
7-0	PINMMRx_7_0	R/W	1h	Each of these byte-fields control the functionality on a given ball/pin. Please refer to Output Multiplexing and Control table for a list of multiplexed signals. This list is sorted by the control register used.

4.8 Output Multiplexing and Control

The device output terminals are shared between several different functions. The controls for these outputs are listed in [Table 4-21](#). PINMMR35-PINMMR47 are used to control ePWMx, eCAPx, and eQEPx modules, see [Section 4.5](#) and the device-specific data manual.

Table 4-21. Output Multiplexing and Control

Default Function	Selection Bit	Alternate Function 1	Selection Bit	Alternate Function 2	Selection Bit	Alternate Function 3	Selection Bit	Alternate Function 4	Selection Bit	Alternate Function 5	Selection Bit
GIOB[3]	PINMMR0[0]	USB2.RCV	PINMMR0[1]	USB_FUNC.RXDI	PINMMR0[2]	RESERVED	PINMMR0[3]	RESERVED	PINMMR0[4]		
GIOA[0]	PINMMR0[8]	USB2.VP	PINMMR0[9]	USB_FUNC.RXDPI	PINMMR0[10]	RESERVED	PINMMR0[11]	RESERVED	PINMMR0[12]	RESERVED	–
MIBSPI3NCS[3]	PINMMR0[16]	I2C_SCL	PINMMR0[17]	N2HET1[29]	PINMMR0[18]	nTZ1	PINMMR0[19]	RESERVED	PINMMR0[20]	RESERVED	–
MIBSPI3NCS[2]	PINMMR0[24]	I2C_SDA	PINMMR0[25]	N2HET1[27]	PINMMR0[26]	nTZ2	PINMMR0[27]	RESERVED	PINMMR0[28]	RESERVED	–
GIOA[1]	PINMMR1[0]	USB2.VM	PINMMR1[1]	USB_FUNC.RXDMI	PINMMR1[2]	RESERVED	PINMMR1[3]	RESERVED	PINMMR1[4]	RESERVED	–
N2HET1[11]	PINMMR1[8]	MIBSPI3NCS[4]	PINMMR1[9]	N2HET2[18]	PINMMR1[10]	USB2. OVERCURRENT	PINMMR1[11]	USB_FUNC. VBUSI	PINMMR1[12]	EPWM1SYNCO	PINMMR1[13]
GIOA[2]	PINMMR2[0]	USB2.TXDAT	PINMMR2[1]	USB_FUNC.TXDO	PINMMR2[2]	N2HET2[0]	PINMMR2[3]	EQEP2I	PINMMR2[4]	RESERVED	–
GIOA[3]	PINMMR2[16]	N2HET2[2]	PINMMR2[17]	RESERVED	PINMMR2[18]	RESERVED	PINMMR2[19]	RESERVED	PINMMR2[20]	RESERVED	–
GIOA[5]	PINMMR2[24]	EXTCLKIN	PINMMR2[25]	EPWM1A	PINMMR2[26]	RESERVED	PINMMR2[27]	RESERVED	PINMMR2[28]	RESERVED	–
N2HET1[22]	PINMMR3[8]	USB2.TXSE0	PINMMR3[9]	USB_FUNC.SE0O	PINMMR3[10]	RESERVED	PINMMR3[11]	RESERVED	PINMMR3[12]	RESERVED	–
GIOA[6]	PINMMR3[16]	N2HET2[4]	PINMMR3[17]	EPWM1B	PINMMR3[18]	RESERVED	PINMMR3[19]	RESERVED	PINMMR3[20]	RESERVED	–
GIOA[7]	PINMMR4[0]	N2HET2[6]	PINMMR4[1]	EPWM2A	PINMMR4[2]	RESERVED	PINMMR4[3]	RESERVED	PINMMR4[4]	RESERVED	–
N2HET1[01]	PINMMR4[16]	SPI4NENA	PINMMR4[17]	USB2.TXEN	PINMMR4[18]	USB_FUNC. PUENO	PINMMR4[19]	N2HET2[8]	PINMMR4[20]	EQEP2A	PINMMR4[21]
N2HET1[03]	PINMMR4[24]	SPI4NCS[0]	PINMMR4[25]	USB2.SPEED	PINMMR4[26]	USB_FUNC. PUENON	PINMMR4[27]	N2HET2[10]	PINMMR4[28]	EQEP2B	PINMMR4[29]
N2HET1[0]	PINMMR5[0]	SPI4CLK	PINMMR5[1]	EPWM2B	PINMMR5[2]	RESERVED	PINMMR5[3]	RESERVED	PINMMR5[4]	RESERVED	–
N2HET1[02]	PINMMR5[8]	SPI4SIMO	PINMMR5[9]	EPWM3A	PINMMR5[10]	RESERVED	PINMMR5[11]	RESERVED	PINMMR5[12]	RESERVED	–
N2HET1[05]	PINMMR5[16]	SPI4SOMI	PINMMR5[17]	N2HET2[12]	PINMMR5[18]	EPWM3B	PINMMR5[19]	RESERVED	PINMMR5[20]	RESERVED	–
N2HET1[07]	PINMMR6[0]	USB2.PORTPOWER	PINMMR6[1]	USB_FUNC.GZO	PINMMR6[2]	N2HET2[14]	PINMMR6[3]	EPWM7B	PINMMR6[4]	RESERVED	–
EMIF_DATA[11]	PINMMR6[8]	EMIF_DATA[11]	PINMMR6[9]	RESERVED	PINMMR6[10]	RESERVED	PINMMR6[11]	RESERVED	PINMMR6[12]	RESERVED	–
N2HET1[09]	PINMMR6[16]	N2HET2[16]	PINMMR6[17]	USB2.SUSPEND	PINMMR6[18]	USB_FUNC. SUSPENDO	PINMMR6[19]	EPWM7A	PINMMR6[20]	RESERVED	–
MIBSPI3NCS[1]	PINMMR7[8]	N2HET1[25]	PINMMR7[9]	MDCLK	PINMMR7[10]	RESERVED	PINMMR7[11]	RESERVED	PINMMR7[12]	RESERVED	–
N2HET1[06]	PINMMR7[16]	SCIRX	PINMMR7[17]	EPWM5A	PINMMR7[18]	RESERVED	PINMMR7[19]	RESERVED	PINMMR7[20]	RESERVED	–
N2HET1[13]	PINMMR8[0]	SCITX	PINMMR8[1]	EPWM5B	PINMMR8[2]	RESERVED	PINMMR8[3]	RESERVED	PINMMR8[4]	RESERVED	–
MIBSPI1NCS[2]	PINMMR8[8]	N2HET1[19]	PINMMR8[9]	MDIO	PINMMR8[10]	RESERVED	PINMMR8[11]	RESERVED	PINMMR8[12]	RESERVED	–
N2HET1[15]	PINMMR8[16]	MIBSPI1NCS[4]	PINMMR8[17]	ECAP1	PINMMR8[18]	RESERVED	PINMMR8[19]	RESERVED	PINMMR8[20]	RESERVED	–
MIBSPI3NENA	PINMMR9[8]	MIBSPI3NCS[5]	PINMMR9[9]	N2HET1[31]	PINMMR9[10]	EQEP1B	PINMMR9[11]	RESERVED	PINMMR9[12]	RESERVED	–
MIBSPI3NCS[0]	PINMMR9[16]	AD2EVT	PINMMR9[17]	GIOB[2]	PINMMR9[18]	EQEP1I	PINMMR9[19]	RESERVED	PINMMR9[20]	RESERVED	–
MIBSPI1NCS[3]	PINMMR9[24]	N2HET1[21]	PINMMR9[25]	RESERVED	PINMMR9[26]	RESERVED	PINMMR9[27]	RESERVED	PINMMR9[28]	RESERVED	–
AD1EVT	PINMMR10[0]	MII_RX_ER	PINMMR10[1]	RMII_RX_ER	PINMMR10[2]	RESERVED	PINMMR10[3]	RESERVED	PINMMR10[4]	RESERVED	–
EMIF_nCS[0]	PINMMR10[16]	RESERVED	PINMMR10[17]	N2HET2[7]	PINMMR10[18]	RESERVED	PINMMR10[19]	RESERVED	PINMMR10[20]	RESERVED	–
EMIF_nCS[3]	PINMMR11[0]	RESERVED	PINMMR11[1]	N2HET2[9]	PINMMR11[2]	RESERVED	PINMMR11[3]	RESERVED	PINMMR11[4]	RESERVED	–
N2HET1[24]	PINMMR11[24]	MIBSPI1NCS[5]	PINMMR11[25]	MII_RXD[0]	PINMMR11[26]	RMII_RXD[0]	PINMMR11[27]	RESERVED	PINMMR11[28]	RESERVED	–
N2HET1[26]	PINMMR12[0]	MII_RXD[1]	PINMMR12[1]	RMII_RXD[1]	PINMMR12[2]	RESERVED	PINMMR12[3]	RESERVED	PINMMR12[4]	RESERVED	–

Table 4-21. Output Multiplexing and Control (continued)

Default Function	Selection Bit	Alternate Function 1	Selection Bit	Alternate Function 2	Selection Bit	Alternate Function 3	Selection Bit	Alternate Function 4	Selection Bit	Alternate Function 5	Selection Bit
MIBSPI1NENA	PINMMR12[16]	N2HET1[23]	PINMMR12[17]	MII_RXD[2]	PINMMR12[18]	USB1.VP	PINMMR12[19]	ECAP4	PINMMR12[20]	RESERVED	–
MIBSPI5NENA	PINMMR12[24]	RESERVED	PINMMR12[25]	MII_RXD[3]	PINMMR12[26]	USB1.VM	PINMMR12[27]	MIBSPI5SOMI[1]	PINMMR12[28]	ECAP5	PINMMR12[29]
MIBSPI5SOMI[0]	PINMMR13[0]	RESERVED	PINMMR13[1]	MII_TXD[0]	PINMMR13[2]	RMII_TXD[0]	PINMMR13[3]	RESERVED	PINMMR13[4]	RESERVED	–
MIBSPI5SIMO[0]	PINMMR13[8]	RESERVED	PINMMR13[9]	MII_TXD[1]	PINMMR13[10]	RMII_TXD[1]	PINMMR13[11]	MIBSPI5SOMI[2]	PINMMR13[12]	RESERVED	–
MIBSPI5CLK	PINMMR13[16]	RESERVED	PINMMR13[17]	MII_TXEN	PINMMR13[18]	RMII_TXEN	PINMMR13[19]	RESERVED	PINMMR13[20]	RESERVED	–
MIBSPI1NCS[0]	PINMMR13[24]	MIBSPI1SOMI[1]	PINMMR13[25]	MII_TXD[2]	PINMMR13[26]	USB1.RCV	PINMMR13[27]	ECAP6	PINMMR13[28]	RESERVED	–
N2HET1[08]	PINMMR14[0]	MIBSPI1SIMO[1]	PINMMR14[1]	MII_TXD[3]	PINMMR14[2]	USB1.OVERCURRENT	PINMMR14[3]	RESERVED	PINMMR14[4]	RESERVED	–
N2HET1[28]	PINMMR14[8]	MII_RXCLK	PINMMR14[9]	RMII_REFCLK	PINMMR14[10]	MII_RX_AVCLK4	PINMMR14[11]	RESERVED	PINMMR14[12]	RESERVED	–
EMIF_nWE	PINMMR14[16]	EMIF_RNW	PINMMR14[17]	RESERVED	PINMMR14[18]	RESERVED	PINMMR14[19]	RESERVED	PINMMR14[20]	RESERVED	–
EMIF_BA[1]	PINMMR14[24]	N2HET2[5]	PINMMR14[25]	RESERVED	PINMMR14[26]	RESERVED	PINMMR14[27]	RESERVED	PINMMR14[28]	RESERVED	–
N2HET1[10]	PINMMR17[0]	MII_TX_CLK	PINMMR17[1]	USB1.TXEN	PINMMR17[2]	MII_TX_AVCLK4	PINMMR17[3]	nTZ3	PINMMR17[4]	RESERVED	–
N2HET1[12]	PINMMR17[16]	MII_CRS	PINMMR17[17]	RMII_CRS_DV	PINMMR17[18]	RESERVED	PINMMR17[19]	RESERVED	PINMMR17[20]	RESERVED	–
N2HET1[14]	PINMMR18[8]	USB1.TXSE0	PINMMR18[9]	RESERVED	PINMMR18[10]	RESERVED	PINMMR18[11]	RESERVED	PINMMR18[12]	RESERVED	–
GIOB[0]	PINMMR18[24]	USB1.TXDAT	PINMMR18[25]	RESERVED	PINMMR18[26]	RESERVED	PINMMR18[27]	RESERVED	PINMMR18[28]	RESERVED	–
N2HET1[30]	PINMMR19[8]	MII_RX_DV	PINMMR19[9]	USB1.SPEED	PINMMR19[10]	EQEP2S	PINMMR19[11]	RESERVED	PINMMR19[12]	RESERVED	–
MIBSPI1NCS[1]	PINMMR20[16]	N2HET1[17]	PINMMR20[17]	MII_COL	PINMMR20[18]	USB1.SUSPEND	PINMMR20[19]	EQEP1S	PINMMR20[20]	RESERVED	–
EMIF_ADDR[1]	PINMMR21[0]	N2HET2[3]	PINMMR21[1]	RESERVED	PINMMR21[2]	RESERVED	PINMMR21[3]	RESERVED	PINMMR21[4]	RESERVED	–
GIOB[1]	PINMMR21[8]	USB1.PORTPOWER	PINMMR21[9]	RESERVED	PINMMR21[10]	RESERVED	PINMMR21[11]	RESERVED	PINMMR21[12]	RESERVED	–
EMIF_ADDR[0]	PINMMR22[0]	N2HET2[1]	PINMMR22[1]	RESERVED	PINMMR22[2]	RESERVED	PINMMR22[3]	RESERVED	PINMMR22[4]	RESERVED	–
EMIF_ADDR[7]	PINMMR22[8]	RESERVED	PINMMR22[9]	N2HET2[13]	PINMMR22[10]	RESERVED	PINMMR22[11]	RESERVED	PINMMR22[12]	RESERVED	–
EMIF_ADDR[6]	PINMMR22[16]	RESERVED	PINMMR22[17]	N2HET2[11]	PINMMR22[18]	RESERVED	PINMMR22[19]	RESERVED	PINMMR22[20]	RESERVED	–
EMIF_ADDR[8]	PINMMR22[24]	RESERVED	PINMMR22[25]	N2HET2[15]	PINMMR22[26]	RESERVED	PINMMR22[27]	RESERVED	PINMMR22[28]	RESERVED	–
EMIF_ADDR[8]	PINMMR23[0]	RESERVED	PINMMR23[1]	N2HET2[15]	PINMMR23[2]	RESERVED	PINMMR23[3]	RESERVED	PINMMR23[4]	RESERVED	–
MIBSPI5NCS[0]	PINMMR27[0]	RESERVED	PINMMR27[1]	EPWM4A	PINMMR27[2]	RESERVED	PINMMR27[3]	RESERVED	PINMMR27[4]	RESERVED	–
SPI2NENA	PINMMR29[0]	SPI2NCS[1]	PINMMR29[1]	RESERVED	PINMMR29[2]	RESERVED	PINMMR29[3]	RESERVED	PINMMR29[4]	RESERVED	–
N2HET1[04]	PINMMR33[0]	EPWM4B	PINMMR33[1]	RESERVED	PINMMR33[2]	RESERVED	PINMMR33[3]	RESERVED	PINMMR33[4]	RESERVED	–
MIBSPI3SOMI[0]	PINMMR33[8]	AWM_EXT_ENA	PINMMR33[9]	ECAP2	PINMMR33[10]	RESERVED	PINMMR33[11]	RESERVED	PINMMR33[12]	RESERVED	–
MIBSPI3SIMO[0]	PINMMR33[16]	AWM_EXT_SEL[0]	PINMMR33[17]	ECAP3	PINMMR33[18]	RESERVED	PINMMR33[19]	RESERVED	PINMMR33[20]	RESERVED	–
MIBSPI3CLK	PINMMR33[24]	AWM_EXT_SEL[1]	PINMMR33[25]	EQEP1A	PINMMR33[26]	RESERVED	PINMMR33[27]	RESERVED	PINMMR33[28]	RESERVED	–
N2HET1[16]	PINMMR34[0]	EPWM1SYNCl	PINMMR34[1]	EPWM1SYNCO	PINMMR34[2]	RESERVED	PINMMR34[3]	RESERVED	PINMMR34[4]	RESERVED	–
N2HET1[18]	PINMMR34[8]	EPWM6A	PINMMR34[9]	RESERVED	PINMMR34[10]	RESERVED	PINMMR34[11]	RESERVED	PINMMR34[12]	RESERVED	–
N2HET1[20]	PINMMR34[16]	EPWM6B	PINMMR34[17]	RESERVED	PINMMR34[18]	RESERVED	PINMMR34[19]	RESERVED	PINMMR34[20]	RESERVED	–